Brigham Young University
BYU ScholarsArchive

Faculty Publications

2013-03-25

A Multiplicative Extended Kalman Filter for Relative Rotorcraft Navigation

Robert C. Leishman

Timothy W. McLain
mclain@byu.edu

Follow this and additional works at: https://scholarsarchive.byu.edu/facpub

Part of the Mechanical Engineering Commons

Original Publication Citation

Robert C. Leishman and Timothy W. McLain. "Multiplicative Extended Kalman Filter for Relative Rotorcraft Navigation", Journal of Aerospace Information Systems, Vol. 12, Special Section on Estimation and Information Theory Application for Resilient and Distributed Operation of Aerospace Systems (2015), pp. 728-744. doi: 10.2514/1.I010236

BYU ScholarsArchive Citation

Leishman, Robert C. and McLain, Timothy W., "A Multiplicative Extended Kalman Filter for Relative Rotorcraft Navigation" (2013). *Faculty Publications*. 1299.
https://scholarsarchive.byu.edu/facpub/1299

This Report is brought to you for free and open access by BYU ScholarsArchive. It has been accepted for inclusion in Faculty Publications by an authorized administrator of BYU ScholarsArchive. For more information, please contact ellen_amatangelo@byu.edu.

A Multiplicative Extended Kalman Filter for Relative Rotorcraft Navigation

Robert C. Leishman, *Student Member, IEEE,*
Timothy W. McLain, *Senior Member, IEEE,*

***Abstract*—In this article we detail the fundamentals of a new approach to GPS-denied navigation for aerial vehicles in confined indoor environments. We depart from the common practice of navigating within a globally referenced map, and instead keep the position and yaw states *relative* to the current node in the map. The approach combines elements of graph SLAM with a multiplicative extended Kalman filter (MEKF). The filter provides quality state estimates at a fast rate and a graph SLAM algorithm maintains a pose graph. We provide specific details for the relative MEKF. We verify the relative estimation approach with hardware flight test results accompanied by comparisons to motion capture truth. We also provide flight results with estimates in the control loop.**

***Index Terms*—Multiplicative EKF, Sensor Fusion, GPS-Denied Navigation, Quaternions, Quadrotor, Indoor Flight.**

I. Introduction

Sensor fusion and localization are important aspects of navigation in unknown, GPS-denied environments, where global measurements and a priori map information are unavailable. Aerial flight using small aircraft like quadrotors is an additional challenge because of limited payload capacity for onboard sensors and computational resources, and the need to control the vehicle's fast dynamics using the state estimates. Unlike ground robots, quadrotors cannot afford to pause in one place until complex algorithms finish processing and state estimates converge for navigation to continue. Accurate and fast state estimates are critical to maintain control of the vehicle and provide quality sensor information.

To date, successful implementations providing GPS-denied autonomous quadrotor flight, like those outlined in [1], [2], [3], [4], [5], [6], [7], [8], typically follow a general approach for localization and mapping. This approach, outlined in [1], enabled the first fully-autonomous quadrotor with the ability to fly using only onboard sensors in a priori unknown environments. Using this original approach the autonomous vehicle must: estimate motion using exteroceptive sensors, fuse motion estimates with other sensor information to form global state estimates, and employ some type of simultaneous localization and mapping (SLAM) to provide loop closure and global consistency.

Incremental motion of the vehicle is computed with an exteroceptive sensor and a motion estimation algorithm. The original approach for GPS-denied autonomous quadrotor flight utilized a planar laser scanner with a fast scan-matching algorithm [1]. A similar method is used in [3]. Planar laser-scanner implementations, however, require strict assumptions regarding the nature of the environment. Six-degree-of-freedom (6DoF) motion estimation using machine vision, as used in the approaches in [2], [4], [7], [9], is advantageous due to a camera sensor's low cost, low power requirements, and light weight. Additionally, high-quality solutions can be obtained using fewer assumptions about the environment. Vision is, however, limited due to requirements for appropriate lighting and a sufficient number of features in the environment. Visual implementations usually employ a version of visual odometry (VO) [10], although a modified version of parallel tracking and mapping (PTAM) [11] is used in [6].

R. Leishman is a Ph.D. candidate with the Department of Mechanical Engineering, Brigham Young University, Provo, UT, 84602 USA e-mail: rleish@gmail.com.

T. McLain is a professor in the Department of Mechanical Engineering, Brigham Young University, Provo, UT, 84602 USA

Manuscript received March 2013

We note that the exteroceptive motion estimates are relative measurements between images or scans. Various approaches are employed to convert the relative measurements into the requisite global form. One method is to sum all previous motion estimates and form a psuedo-global measurement [1]. Another is to treat the measurement as an average velocity over the duration between sensor messages [4], [7]. Stochastic cloning [8], [9], [12] is a more valid option as the state is cloned to more appropriately handle the cross-correlations that are necessary to form a correct global measurement update.

Sensor fusion combines exteroceptive motion measurements with IMU information for improved global state estimates at the fast rate needed for feedback control of the quadrotor. Typically some form of an extended Kalman filter (EKF) is employed and the accelerometer and gyroscope measurements are used in the filter propagation step [1], [3], [4]. In contrast, the approach detailed in [7] uses a simple low-pass filter for sensor fusion. One difficultly noted in many of these approaches is the lack of a good velocity measurement to aid in the sensor fusion [13], [1], [3], [7]. This issue is discussed in [14] and an improved quadrotor dynamic model is shown to provide body-fixed velocity information using only accelerometer measurements.

Global state estimates of the vehicle will drift if only relative motion and IMU measurements are used. SLAM is a solution that provides the capability to maintain a map and to perform loop closure to eliminate drift and obtain globally consistent state estimates. The use of the pose graph and nonlinear optimization tools from the graph-SLAM paradigm [15], [16] and visual place recognition to find possible loop closures [17] is increasingly common. Graph SLAM provides flexibility in working with the sensor fusion and ease with which place recognition loop closures may be applied to the map. Visual place recognition algorithms recognize previously visited locations in the map using only visual information. Because of the need for globally metric information, a majority of the approaches require feedback from the SLAM algorithms as a measurement in the sensor fusion.

One problem with the general approach, which was noted in [5], is the requirement of globally metric information for the maps of the environment and states of the vehicle. Requiring global states incurs difficulties such as the need for additional states to incorporate relative position measurements [9], [8], waiting periods for global consistency [2], dependence on computationally-expensive optimization and loop closure feedback for localization [4], [7] and logic to accommodate large jumps in pose when loop closures are applied [8].

In [18], [19], the authors propose that a vehicle should navigate using a *relative* formulation of the vehicle state, rather than a global one. Similar to the general approach, they use a combination of graph SLAM and an EKF to provide mapping and sensor fusion. The map is a pose graph, with images from the onboard camera as key components of the nodes. The EKF provides estimates at the high rate required for feedback control of the vehicle. The difference is that the position and yaw states of the EKF are defined with respect to the current node in the map, rather than to a global origin. Relative state information affords many advantages, such as the ability to directly utilize relative exteroceptive measurements, elimination of required feedback to the filter from computationally-expensive SLAM algorithms, easy creation of map edges using the filter state and covariance, and flexible use of global information.

The main contribution of this article is the development of a multiplicative extended Kalman filter (MEKF) that uses an improved rotorcraft model [14] to provide relative, rather than global, state estimates. Another contribution is a more detailed development of the relative navigation approach originally described in [19] and the

relative filtering approach described in [18]. Additionally, we verify the results of the relative estimation approach with hardware flight tests accompanied by comparisons to motion capture truth data. We also provide flight results with estimates in the control loop.

We utilize a hexacopter vehicle from MikroKopter, pictured in Figure 1, as the platform for experiments. An IMU, sonar altimeter, and front-facing RGB-D camera are the only sensors used for the sensor fusion and mapping. All computation is completed onboard using a small computer to perform the necessary processing. .

Fig. 1. The Mikrokopter hexacopter that we use to carry out the experiments. The only sensors we utilize are an altimeter, IMU, and front-facing RGB-D camera.

The remainder of the article is outlined as follows. We explain the approach to relative navigation in Section II. The model and MEKF are derived in Section III. The results are described in Section IV. And we summarize the work in Section V.

II. Relative Navigation Approach

Relative navigation refers to navigation with respect to a local reference frame. We propose that the local frame change as the vehicle moves through the environment, establishing a topological representation of the world using a pose graph [20]. The changes in the local frame occur based on the needs of a robust VO algorithm, described in [21]. The algorithm is *keyframe* based. Instead of comparing consecutive images, each current image is compared to a reference image, called a keyframe, to obtain the 6DoF change in pose. New keyframes are declared when the vehicle has moved further than a predetermined threshold from the previous keyframe and the overlap between images becomes too small for reliable matching. The local coordinate frames, with respect to which the vehicle navigates, are derived from the keyframes.

The map in Figure 2 illustrates the relative topological approach. The VO algorithm initializes a keyframe at node 1 and an edge is added between the global frame and the node frame once this information is known. The filter estimates the position and yaw states of the vehicle with respect to the local coordinate frame at node 1 as the vehicle travels. When the VO requires a new keyframe to maintain good performance, a new keyframe and node are declared at pose 2. An edge is added to the map using the relative states and covariance in the MEKF. The navigation then continues with respect to node 2 by marginalizing out the old relative states and augmenting the state vector with new ones. This process continues as the vehicle moves through the environment, with new keyframes and nodes being declared as necessary and the MEKF changing the relative states each time a new keyframe is declared. As current images are compared to a keyframe, the position estimates will not drift as long as the keyframe does not change. A vector chain of edges connects the hexacopter to the global reference frame. Global position and yaw for the vehicle can be estimated by summing all the edges and the current state.

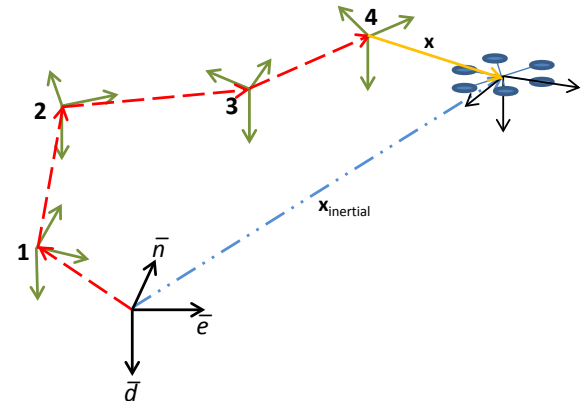

Fig. 2. Relative navigation using nodes and edges. As the vehicle flies through the environment, nodes are created using the VO keyframes and edges are defined between them using the relative states of the MEKF. The vehicle state is relative to the current node (node 4). The global state of the vehicle can be found by summing the edges and the current state.

This relative navigation approach has several key advantages: direct use of sensor information for state updates, straightforward creation of map edges using the filter state and covariance, and flexible use of global information. We discuss below how each of these is helpful for autonomous aerial navigation.

Exteroceptive sensors provide relative information. In particular, the VO provides the change in 6DoF pose between the current and keyframe images. By expressing the VO result in the node coordinate frame, the position and attitude are updated directly in the filter. This simplification eliminates the need for additional states in the filter and does not require VO measurements to update the velocity states.

Defining edges between consecutive nodes is a simple matter of saving the relative portions of the state and covariance just before a new node is created. The covariances from each edge could be combined and then used for a confidence measure of the current global position. For example, a path planning algorithm might use the combined covariances of the edges to indicate when estimates have drifted sufficiently to warrant a planned loop closure.

Our proposed relative approach offers more flexibility than a globally-based method. A front-end subsystem provides the critical processes to keep the hexacopter flying, including the VO, sensor fusion, control, and obstacle avoidance. All the components in the front end are based in the relative coordinate frame explained above. A back-end subsystem will maintain globally consistent information, including a global map and high-level, global objectives when desired. The only information provided by the back end or by human operators are goal locations expressed in the current node reference frame. This is in contrast to other solutions that have been developed that require feedback from the computationally expensive visual place recognition and optimization components. Additional details on the architecture of the approach are found in [19].

This separation between the time-critical front end and globally consistent back end provides the flexibility mentioned. The system can fly reliably either with or without loop-closure constraints that constrain drift, and with or without global optimization. This is possible because the local navigation and control take place regardless of global changes within the map. Without loop closure it is clear that the map will drift and not remain globally consistent. The relative relationships between nodes, however, maintain locally consistent topological and metric relationships between saved locations. Therefore, the map could be traversed, even back to the starting location, by using these relative relationships. This is also true when loop closure

constraints are available and global optimization is not; we could then pursue a consistent but purely relative topological approach similar to that of [22]. Finally, by enabling both loop closure and global optimization we could mimic the typical SLAM approach that provides globally consistent metric information of the environment.

A. Node Frame

The nodes in the pose graph contain keyframe images and they represent locations in the environment where those images are taken. Each node has a local coordinate frame defined, with respect to which the vehicle navigates through the environment. The local frame is formed using the global $\vec{n}$-$\vec{e}$ plane and the body-fixed coordinate frame at the instant in time when the keyframe image is taken. This particular body frame is designated the *reference* body-fixed coordinate frame.

The node frame for the j-th node is defined by unit vectors in the front, right, and down directions: $\vec{f}_j$, $\vec{r}_j$, and $\vec{d}_j$. The $\vec{d}_j$ direction is parallel to the global $\vec{d}$ direction. The $\vec{f}_j$ direction is defined by the projection of the reference body $\vec{b}_x$ axis onto a plane parallel to the global $\vec{n}$-$\vec{e}$ plane, as shown in Figure 3. The $\vec{r}_j$ direction is defined to make a right-hand coordinate system. This representation preserves the heading of the vehicle, which corresponds with the direction the image was taken, and maintains a global down direction, which keeps the roll and pitch defined correctly.

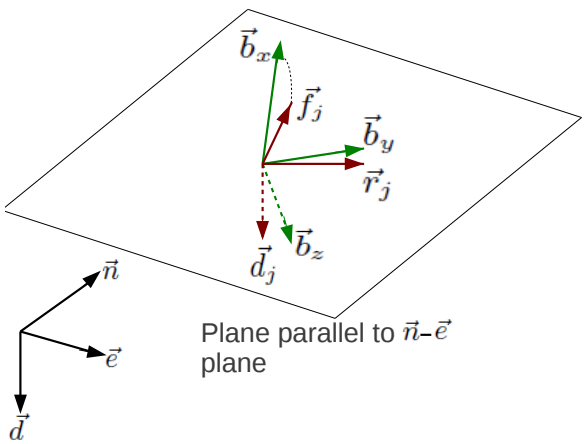

Fig. 3. Relationship between the global (black), body (green), and node (red) frames at the instant the j-th keyframe image is taken. The $\vec{f}_j$ direction is defined by the projection of the reference body $\vec{b}_x$ axis onto a plane parallel to the global $\vec{n}$-$\vec{e}$ plane.

The relationship between the keyframe and the local node frame is necessary for correctly expressing VO measurement updates. Points defined in the keyframe's camera coordinate frame are expressed in the node coordinate frame using two rotations, $\mathcal{R}_c^b$ and $\mathcal{R}_{br}^n$, and a translation, $\mathbf{p}^b$. The rotation $\mathcal{R}_c^b$ and translation $\mathbf{p}^b$ define the static transformation between the camera frame and the body-fixed frame that is obtained through calibration. $\mathcal{R}_c^b$ rotates points from the camera frame into the body-fixed frame and $\mathbf{p}^b$ are the coordinates of the camera focal point in the body-fixed coordinate frame. These values only change when the location or orientation of the camera or IMU changes on the platform.

The second rotation, $\mathcal{R}_{br}^n$, defines the relationship between the reference body-fixed frame and the node frame. The reference body-fixed frame is created by saving the roll and pitch angles of the vehicle, ϕ_j and θ_j, at the instant in time the image is taken. The matrix $\mathcal{R}_{br}^n(\phi_j, \theta_j)$ rotates points from the body-fixed frame into the node j coordinate frame. Combined together, the transformation $\mathcal{R}_b^n\left(\mathcal{R}_c^b \mathbf{p}^{cr} + \mathbf{p}^b\right)$ transforms points from the keyframe coordinate frame to the node coordinate frame.

B. Map

The map used in this work is a collection of nodes and edges in a relative topological pose graph. New nodes are created with each new keyframe. Edges are added between temporally and spatially consecutive keyframes.

A node is described, ultimately, by a keyframe RGB and depth image pair. Attached to the keyframe pair are the estimates of relative and global position and orientation, yet the image encodes the true instantaneous location of the vehicle. The node coordinate frame, defined above, enables navigation relative to the node.

Edges in the graph represent the estimated relative transformations between nodes. We currently only consider edges from the odometry, but we plan to soon include other constraints, such as those from visual recognition loop closures and intermittent GPS measurements. The odometry edges are created using the MEKF, based on the measurements from the VO algorithm. When a new node is received by the estimator from the VO, the old relative portions of the state and covariance are marginalized out and saved as the edge between the old and new nodes.

III. Multiplicative Extended Kalman Filter

In contrast to the work in [18], we now use unit-length quaternions to represent the attitude of the vehicle. A quaternion is parameterized by a scalar and a vector. We utilize the standard Hamiltonian form of the quaternion [23]. A unit-length quaternion represents an over-parameterization of the attitude and should not be used directly in an EKF framework because of singularity problems that can occur in the covariance matrix. Many generic estimation methods have been developed to handle this difficulty and are available and commonly used in practice [24]. We chose to derive a MEKF [25], [26] for our relative hexacopter system.

The MEKF is an indirect EKF, which means that the error in the state $\Delta\mathbf{x}$ and the covariance of the error are maintained in the filter rather than the best estimate $\hat{\mathbf{x}}$ and error covariance. The name *Multiplicative* was coined as the error in the quaternions is computed using a quaternion product $\otimes$ instead of subtraction (i.e., $\delta q(\theta) = q \otimes \hat{q}^{-1}$) [24]. The three-dimensional error vector θ, obtained from the error quaternion using a small angle approximation, is maintained in the error state. This results in a minimal representation that is safe to use in calculating the covariance. We also use a continuous-discrete representation [27] for our implementation of the MEKF.

In this implementation we do not directly maintain an estimate of the error state in the filter at each timestep and therefore do not maintain a true indirect filter. Instead, we maintain the best estimate $\hat{\mathbf{x}}$ and the covariance $\mathbf{P}$ of the error state $\Delta\mathbf{x}$ in the filter. Details of the implementation are given below.

A. State Dynamics

We model the hexacopter with the nonlinear equations

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}), \tag{1}$$

$$\mathbf{y}_i = \mathbf{h}_i(\mathbf{x}, \mathbf{u}), \quad 1 \leq i \leq p, \tag{2}$$

where $\mathbf{h}_i$ is the ith measurement function, and the vector $\mathbf{u}$ represents the inputs that drive the evolution of the estimated states. The inputs to the model are simply the gyroscope measurements and the $\vec{b}_z$ accelerometer

$$\mathbf{u} = \begin{bmatrix} p_{gyro} & q_{gyro} & r_{gyro} & z_{accel} \end{bmatrix}^\top. \tag{3}$$

The true states $\mathbf{x}$ of the multirotor are defined with respect to the current node, node j,

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}^{n\top} & \mathbf{q}_n^{b\top} & \mathbf{v}^{b\top} & \beta^\top & \alpha^\top & \mathbf{q}_c^{b\top} & \mathbf{p}^{b\top} \end{bmatrix}^\top. \quad (4)$$

The relative position vector $\mathbf{p}^n$, made up of f_j, r_j and d_j, is the displacement of the body with respect to node j. The quaternion $\mathbf{q}_n^b$, with components q_x, q_y, q_z and q_w, expresses the attitude of the body frame with respect to the node frame. The component q_z is relative to the current node. $\mathbf{v}^b$ is the body frame velocity vector, composed of u, v and w. The gyroscope bias vector is β. We only estimate the accelerometer biases in the body $\vec{b}_x$ and $\vec{b}_y$ directions in α. The last two parameters in (4) represent the transformation from the body coordinate frame to the camera coordinate frame and can be optionally included in the state. $\mathbf{q}_c^b$ is the quaternion form of the the rotation $\mathcal{R}_c^b$, which rotates points from the camera coordinate frame into the body coordinate frame. $\mathbf{p}^b$ are the coordinates of the camera focal point expressed in the body coordinate frame. Once refinements to the calibration parameters are obtained, these estimates can be saved as constants and then removed from the state vector. The estimated state vector is denoted $\hat{\mathbf{x}}$.

The nonlinear equations in (1) are

$$\dot{\mathbf{p}}^n = \mathbf{q}_n^{b\,-1} \otimes \mathbf{v}^b \otimes \mathbf{q}_n^b = \mathcal{R}^\top(\mathbf{q}_n^b)\mathbf{v}^b, \quad (5)$$

$$\dot{\mathbf{q}}_n^b = \frac{1}{2}\Omega\left(\mathbf{u}_{(1:3)} - \beta - \eta_\omega\right)\mathbf{q}_n^b, \quad (6)$$

$$\dot{\mathbf{v}}^b = \mathbf{v}^b \times \left(\mathbf{u}_{(1:3)} - \beta - \eta_\omega\right) + \mathbf{q}_n^b \otimes \mathbf{g} \otimes \mathbf{q}_n^{b\,-1} - \frac{1}{m}\mathbf{M}\mathbf{v}^b + \mathbf{u}_{(4)}\vec{d}_j, \quad (7)$$

$$\dot{\beta} = \eta_\beta \quad (8)$$

$$\dot{\alpha} = \eta_\alpha \quad (9)$$

$$\dot{\mathbf{q}}_c^b = \mathbf{0} \quad (10)$$

$$\dot{\mathbf{p}}^b = \mathbf{0}, \quad (11)$$

where η_ω, η_β and η_α are zero-mean, Gaussian processes for the noise in the gyroscope measurements and the random-walk models of the gyroscope and the accelerometer biases. A rotation matrix $\mathcal{R}(\mathbf{q}_x^y)$ from a quaternion $\mathbf{q}_x^y$ is equivalent to the quaternion rotational operator $\mathcal{L}\left(q_x^y\right) = \mathbf{q}_x^y \otimes \mathbf{v} \otimes \mathbf{q}_x^{y\,-1}$, which rotates the vector $\mathbf{v}$, expressed in the frame x, into frame y. The operator

$$\Omega\left(\omega\right) = \begin{bmatrix} 0 & \omega_3 & -\omega_2 & \omega_1 \\ -\omega_3 & 0 & \omega_1 & \omega_2 \\ \omega_2 & -\omega_1 & 0 & \omega_3 \\ -\omega_1 & -\omega_2 & -\omega_3 & 0 \end{bmatrix}$$

assumes that the order of a quaternion it multiplies is $\begin{bmatrix} q_x & q_y & q_z & q_w \end{bmatrix}^\top$. The constant matrix $\mathbf{M}$ is

$$\mathbf{M} = \begin{bmatrix} \mu & 0 & 0 \\ 0 & \mu & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

and the constants $\mathbf{g}$ and μ are the gravity and drag coefficient respectively. An improved model of the hexacopter dynamics, contained in (7), which accounts for the rotor drag with coefficient μ, provides the ability to fully utilize the information contained in the accelerometer measurements. As a consequence, estimation accuracy improves and the requirements for VO or any other exteroceptive measurement updates are reduced [14], [28].

B. Error Dynamics

We now derive the error dynamics using the nonlinear dynamics in (5) through (11). The error dynamics are used to propagate the error covariance matrix $\mathbf{P}$.

Error for all the components of the state, except the quaternions, are defined as

$$\delta\mathbf{a} = \mathbf{a} - \hat{\mathbf{a}}. \quad (12)$$

Error for a quaternion is defined by

$$\delta\mathbf{q} = \mathbf{q} \otimes \hat{\mathbf{q}}^{-1}. \quad (13)$$

Note that for a quaternion or a rotation matrix, the inversion operator is equivalent to the transpose. For a quaternion, this simply involves changing the sign of the vector portion. We then define the attitude error vector $\delta\theta$, using the small angle assumption, as

$$\delta\mathbf{q} \approx \begin{bmatrix} \frac{1}{2}\delta\theta \\ 1 \end{bmatrix} \quad (14)$$

The attitude error vector is maintained in the error state and, like the other error components, is assumed to be small. The length of the error state is reduced by one for each quaternion when compared to the length of the true state. The 20-element relative error state is then

$$\Delta\mathbf{x} = \begin{bmatrix} \delta\mathbf{p}^{n\top} & \delta\theta_n^{b\top} & \delta\mathbf{v}^{b\top} & \delta\beta^\top & \delta\alpha^\top & \delta\theta_c^{b\top} & \delta\mathbf{p}^{b\top} \end{bmatrix}^\top. \quad (15)$$

The error dynamics, computed using (12), (13), and (5) through (11) are as follows. In the derivation we assumed that second-order effects are negligible. We also used the identities $\mathcal{R}(\delta q) \approx \mathbf{I}_{3\times3} - \lfloor\delta\theta\rfloor$, where $\lfloor\ \rfloor$ is the skew-symmetric matrix operator on a vector, and $\lfloor\mathbf{y}\rfloor\,\mathbf{x} = -\lfloor\mathbf{x}\rfloor\,\mathbf{y}$ [25].

$$\dot{\delta\mathbf{p}}_n^b = \mathcal{R}^\top(\hat{\mathbf{q}}_n^b)\delta\mathbf{v}^b - \mathcal{R}^\top(\hat{\mathbf{q}}_n^b)\left\lfloor\hat{\mathbf{v}}^b\right\rfloor\delta\theta_n^b \quad (16)$$

$$\dot{\delta\theta}_n^b = -\,\mathbf{u}_{(1:3)} \times \delta\theta_n^b - \beta - \eta_\omega \quad (17)$$

$$\dot{\delta\mathbf{v}}^b = \delta\mathbf{v}^b \times \left(\mathbf{u}_{(1:3)} - \hat{\beta} - \eta_\omega\right) + \hat{\mathbf{v}}^b \times (-\delta\beta - \eta_\omega) + \left\lfloor\mathcal{R}^\top\left(\hat{\mathbf{q}}_n^b\right)\mathbf{g}\right\rfloor\delta\theta_n^b - \frac{1}{m}\mathbf{M}\delta\mathbf{v} \quad (18)$$

$$\dot{\beta} = \eta_\beta \quad (19)$$

$$\dot{\alpha} = \eta_\alpha \quad (20)$$

$$\dot{\mathbf{q}}_c^b = \mathbf{0} \quad (21)$$

$$\dot{\mathbf{p}}^b = \mathbf{0} \quad (22)$$

Equations (16) through (22) are linearized and result in the following linear model

$$\dot{\Delta\mathbf{x}} = \mathbf{A}\Delta\mathbf{x} + \mathbf{B}\mathbf{u}, \quad (23)$$

where $\mathbf{A}$ is the Jacobian of (16) through (22) with respect to the error state $\Delta\mathbf{x}$ and $\mathbf{B}$ is the Jacobian of (16) through (22) with respect to the input $\mathbf{u}$.

C. Prediction

As was mentioned above, in the filter we maintain the estimated state $\hat{\mathbf{x}}$ and the covariance $\hat{\mathbf{P}}$ of the error state (15). The estimated states are propagated forward by numerically integrating the nonlinear equations of motion, (5) through (11). The estimated covariance is propagated forward by numerically integrating the equation

$$\dot{\hat{\mathbf{P}}} = \mathbf{A}\hat{\mathbf{P}} + \hat{\mathbf{P}}\mathbf{A}^\top + \gamma\left(\mathbf{B}\mathbf{G}\mathbf{B}^\top + \mathbf{Q}\right), \quad (24)$$

where the matrices $\mathbf{A}$ and $\mathbf{B}$ are from (23), γ is a tuning parameter, $\mathbf{G}$ is a diagonal matrix of the measured covariance on the inputs (3) and $\mathbf{Q}$ is the process noise covariance which represents modeling error and disturbances.

In contrast to [18], some diagonal portions of $\mathbf{Q}$ are non-zero and must be found through hand tuning for the best performance. As we mentioned above, second-order terms were neglected in the

derivations of (16) through (18). This results in not being able to completely account for the error.

D. Measurement Updates

We update the filter with measurements of the form in (2). Specifically, altimeter, accelerometer, and VO position and orientation measurements are used. In this implementation we treat the VO measurements updates separately. This is possible because we account for the contribution of the rotational uncertainty in the position covariance when the covariances of the measurements are generated [21].

Each measurement update follows the same procedure. For generality, we will assume that the measurement is $\mathbf{h}$, the measurement from the model is $\hat{\mathbf{h}}$ and the Jacobian of the analytic residual $\Delta\mathbf{h}$ with respect to the *error state* is $\mathbf{H}$.

First we compute the residual $\Delta\mathbf{h}$

$$\Delta\mathbf{h} = \mathbf{h} - \hat{\mathbf{h}}. \tag{25}$$

The covariance of the residual $\mathbf{S}$ is computed using

$$\mathbf{S} = \mathbf{R}_h + \mathbf{H}\hat{\mathbf{P}}\mathbf{H}^\top, \tag{26}$$

where $\mathbf{R}_h$ is the covariance on the measurement model $\hat{\mathbf{h}}$. The Kalman gain $\mathbf{L}$ is

$$\mathbf{L} = \hat{\mathbf{P}}\mathbf{H}^\top\mathbf{S}^{-1}. \tag{27}$$

The correction (or updated error state) $\Delta\hat{\mathbf{x}}^+$ is computed as

$$\Delta\hat{\mathbf{x}}^+ = \mathbf{L}\Delta\mathbf{h}, \tag{28}$$

where the $^+$ notation denotes an updated variable. The covariance is updated using

$$\hat{\mathbf{P}}^+ = (\mathbf{I} - \mathbf{L}\mathbf{H})\,\hat{\mathbf{P}}. \tag{29}$$

After each measurement update, we need to use the correction (28) to update the current state estimate $\hat{\mathbf{x}}$. Using (12), we can update a component $\mathbf{a}$ of the state, which is not a quaternion, as

$$\hat{\mathbf{a}}^+ = \hat{\mathbf{a}} + \delta\hat{\mathbf{a}}^+. \tag{30}$$

To update the quaternions in the state, we create an error quaternion $\delta\mathbf{q}$ from the quaternion error vector $\delta\theta$ and then use the quaternion product to compute the updated quaternion. First, we change the error vector into the vector portion of the error quaternion using

$$\delta\hat{\mathbf{q}}^+_{vec} = \frac{1}{2}\delta\hat{\theta}^+.$$

Next, we create the unit-length error quaternion

$$\delta\hat{\mathbf{q}}^+ = \left[\frac{\delta\hat{\mathbf{q}}^+_{vec}}{\sqrt{1 - \left(\delta\hat{\mathbf{q}}^+_{vec}\right)^\top \delta\hat{\mathbf{q}}^+_{vec}}}\right];$$

or, if $\left(\delta\hat{\mathbf{q}}^+_{vec}{}^\top \delta\hat{\mathbf{q}}^+_{vec}\right) > 1$, using

$$\delta\hat{\mathbf{q}}^+ = \frac{1}{\sqrt{1 + \left(\delta\hat{\mathbf{q}}^+_{vec}\right)^\top \delta\hat{\mathbf{q}}^+_{vec}}} \cdot \begin{bmatrix}\delta\hat{\mathbf{q}}^+_{vec}\\ 1\end{bmatrix}.$$

Finally we can compute the new quaternion for the state estimate using the error quaternion $\delta\mathbf{q}$ and the estimate from the prediction step $\hat{\mathbf{q}}$

$$\hat{\mathbf{q}}^+ = \delta\hat{\mathbf{q}}^+ \otimes \hat{\mathbf{q}} \tag{31}$$

The measurement models for each sensor are outlined below. Each of these is used in the generalized method outlined in (25) through (31) to complete the measurement updates for the filter.

1) Altimeter Measurement Model: The altimeter provides a global measurement of the altitude of the vehicle, assuming flight near hover and a flat floor. We can obtain an estimate of the global altitude using the position in the current state $\hat{\mathbf{p}}$ and the global position $\hat{\mathbf{p}}_{node}$ of the current node with respect to which we are navigating. No rotational transformation is necessary as the global down and node down directions are parallel. To compute the Jacobian $\mathbf{H}_{alt}$ of the residual with respect to the error state (15), we must develop an analytical expression for the residual $\Delta\mathbf{h} = \mathbf{h}_{alt} - \hat{\mathbf{h}}_{alt}$. We have

$$\begin{aligned}\mathbf{h}_{alt} &= \mathbf{p}(3) + \mathbf{p}_{node}(3) + \eta_{alt}\\ \hat{\mathbf{h}}_{alt} &= \hat{\mathbf{p}}(3) + \hat{\mathbf{p}}_{node}(3),\end{aligned}$$

for which the analytic residual is

$$\begin{aligned}\Delta\mathbf{h}_{alt} &= \mathbf{h}_{alt} - \hat{\mathbf{h}}_{alt}\\ &= \mathbf{p}(3) + \mathbf{p}_{node}(3) + \eta_{alt} - (\hat{\mathbf{p}}(3) + \hat{\mathbf{p}}_{node}(3))\\ &= \delta\mathbf{p}(3) + \delta\mathbf{p}_{node}(3) + \eta_{alt}.\end{aligned} \tag{32}$$

The Jacobian $\mathbf{H}_{alt}$ of the residual is trivially

$$\mathbf{H}_{alt} = \begin{bmatrix}\begin{bmatrix}0 & 0 & 1\end{bmatrix} & \mathbf{0}_{1\times3} & \mathbf{0}_{1\times3} & \mathbf{0}_{1\times3} & \mathbf{0}_{1\times2} & \mathbf{0}_{1\times3} & \mathbf{0}_{1\times3}\end{bmatrix}. \tag{33}$$

The covariance of the measurement is $E\left[\eta_{alt}\eta_{alt}^\top\right] = \mathbf{R}_{alt}$.

2) Accelerometer Measurement Model: The body $\vec{b}_x$ and $\vec{b}_y$ axis measurements are modeled as

$$\begin{aligned}\mathbf{h}_{acc} &= -\frac{1}{m}\mathbf{M}\mathbf{v}^b + \alpha + \eta_{acc}\\ \hat{\mathbf{h}}_{acc} &= -\frac{1}{m}\mathbf{M}\hat{\mathbf{v}}^b + \hat{\alpha}.\end{aligned}$$

The analytic residual is

$$\Delta\mathbf{h}_{acc} = -\frac{1}{m}\mathbf{M}\delta\mathbf{v}^b + \delta\alpha + \eta_{acc}. \tag{34}$$

We can then calculate the Jacobian $\mathbf{H}_{acc}$:

$$\mathbf{H}_{acc} = \begin{bmatrix}\mathbf{0}_{3\times3} & -\frac{1}{m}\mathbf{M} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \begin{bmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 0\end{bmatrix} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3}\end{bmatrix}. \tag{35}$$

3) Visual Odometry Position Model: Figure 4 illustrates the chain of transformations involved in the measurement updates for the VO. The current pose in the state is represented by the transformation from the current node frame to the current body frame, $\mathbf{q}^b_n$ and $\mathbf{p}^n$. The calibration terms, $\mathbf{q}^b_c$ and $\mathbf{p}^b$, are found in two places since there is a reference body frame in between the node frame and the reference camera frame. The VO provides the transformation made up of $\mathbf{q}^c_{cr}$ and $\mathbf{p}^c$.

Using Figure 4, we estimate the model of the VO translation $\mathbf{p}^c$ as

$$\begin{aligned}&\mathbf{h}_{vp} =\\ &-\mathcal{L}^c_b\left(\mathbf{q}^{b\top}_c\right)\mathbf{p}^b - \mathcal{L}^c_n\left(\mathbf{q}^b_n\mathbf{q}^{b\top}_c\right)\mathbf{p}^n + \mathcal{L}^c_{br}\left(\mathbf{q}^n_{br}\mathbf{q}^b_n\mathbf{q}^{b\top}_c\right)\mathbf{p}^b + \eta_{vp}\\ &\hat{\mathbf{h}}_{vp} =\\ &-\mathcal{L}^c_b\left(\hat{\mathbf{q}}^{b\top}_c\right)\hat{\mathbf{p}}^b - \mathcal{L}^c_n\left(\hat{\mathbf{q}}^b_n\,\hat{\mathbf{q}}^{b\top}_c\right)\hat{\mathbf{p}}^n + \mathcal{L}^c_{br}\left(\hat{\mathbf{q}}^n_{br}\hat{\mathbf{q}}^b_n\,\hat{\mathbf{q}}^{b\top}_c\right)\hat{\mathbf{p}}^b\end{aligned}$$

The analytic residual begins as

$$\begin{aligned}&\Delta\mathbf{h}_{vp} =\\ &-\mathcal{L}^c_b\left(\mathbf{q}^{b\top}_c\right)\mathbf{p}^b - \mathcal{L}^c_n\left(\mathbf{q}^b_n\mathbf{q}^{b\top}_c\right)\mathbf{p}^n + \mathcal{L}^c_{br}\left(\mathbf{q}^n_{br}\mathbf{q}^b_n\mathbf{q}^{b\top}_c\right)\mathbf{p}^{br} + \eta_{vp} -\\ &\left(-\mathcal{L}^c_b\left(\hat{\mathbf{q}}^{b\top}_c\right)\hat{\mathbf{p}}^b - \mathcal{L}^c_n\left(\hat{\mathbf{q}}^b_n\,\hat{\mathbf{q}}^{b\top}_c\right)\hat{\mathbf{p}}^n + \mathcal{L}^c_{br}\left(\hat{\mathbf{q}}^n_{br}\hat{\mathbf{q}}^b_n\,\hat{\mathbf{q}}^{b\top}_c\right)\hat{\mathbf{p}}^{br}\right).\end{aligned}$$

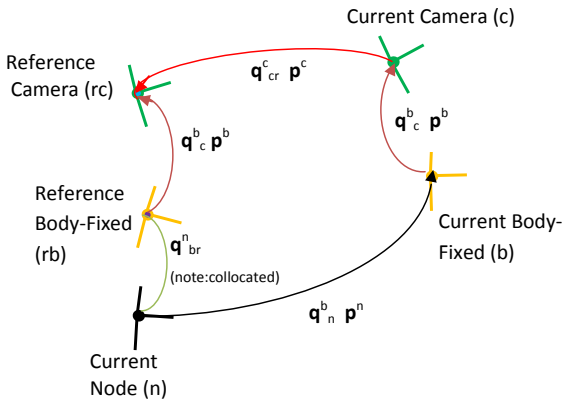

Fig. 4. Vector chain for the view-matching measurement updates. The labels of the frames, as used in the equations below, are noted in parenthesis. Note that the arrows point in the defined direction of the translation vector.

We can switch to a rotation matrix form and replace the truth terms with the estimate and error using (12). Note that the order of the rotation terms must switch when going from quaternions to rotation matrices as we are using the Hamilton representation of the quaternion

$$\begin{aligned}
&\Delta\mathbf{h}_{vp} = \\
&-\mathcal{R}^\top(\delta\mathbf{q}^b_c)\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\left(\delta\mathbf{p}^b + \hat{\mathbf{p}}^b\right) \\
&-\mathcal{R}^\top(\delta\mathbf{q}^b_c)\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\mathcal{R}(\delta\mathbf{q}^b_n)\left(\delta\mathbf{p}^n + \hat{\mathbf{p}}^n\right) \\
&+\mathcal{R}^\top(\delta\mathbf{q}^b_c)\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\mathcal{R}(\delta\mathbf{q}^b_n)\mathcal{R}(\mathbf{q}^n_{br})\left(\delta\mathbf{p}^b + \hat{\mathbf{p}}^b\right) + \eta_{vp} \\
&+\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\hat{\mathbf{p}}^b + \mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\hat{\mathbf{p}}^n - \mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\mathcal{R}(\mathbf{q}^n_{br})\hat{\mathbf{p}}^b.
\end{aligned}$$

Finally, after expanding, removing second-order terms, and simplifying using the approximation $\mathcal{R}(\delta\mathbf{q}^y_x) \approx (\mathbf{I} - \lfloor\delta\theta^y_x\rfloor)$ or $\mathcal{R}^\top(\delta\mathbf{q}^y_x) \approx (\mathbf{I} + \lfloor\delta\theta^y_x\rfloor)$, we have

$$\begin{aligned}
&\Delta\mathbf{h}_{vp} = \\
&-\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\delta\mathbf{p}^b - \left\lfloor\delta\theta^b_c\right\rfloor\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\hat{\mathbf{p}}^b - \mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\delta\mathbf{p}^n \\
&+\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\left\lfloor\delta\theta^b_n\right\rfloor\hat{\mathbf{p}}^n - \left\lfloor\delta\theta^b_c\right\rfloor\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\hat{\mathbf{p}}^n \qquad (36) \\
&+\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\mathcal{R}(\mathbf{q}^n_{br})\delta\mathbf{p}^b - \mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\left\lfloor\delta\theta^b_n\right\rfloor\mathcal{R}(\mathbf{q}^n_{br})\hat{\mathbf{p}}^b + \\
&\left\lfloor\delta\theta^b_c\right\rfloor\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\mathcal{R}(\mathbf{q}^n_{br})\hat{\mathbf{p}}^b.
\end{aligned}$$

The Jacobian $\mathbf{H}_{vp}$ of the analytic residual is

$$\mathbf{H}^\top_{vp} = \qquad (37)$$
$$\begin{bmatrix}
-\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n) \\
-\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\lfloor\hat{\mathbf{p}}^n\rfloor + \mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\left\lfloor\mathcal{R}(\mathbf{q}^n_{br})\hat{\mathbf{p}}^b\right\rfloor \\
\mathbf{0}_{3\times3} \\
\mathbf{0}_{3\times3} \\
\mathbf{0}_{3\times2} \\
\left\lfloor\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\hat{\mathbf{p}}^b\right\rfloor + \left\lfloor\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\hat{\mathbf{p}}^n\right\rfloor - \left\lfloor\mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\mathcal{R}(\mathbf{q}^n_{br})\hat{\mathbf{p}}^b\right\rfloor \\
-\mathcal{R}^\top(\hat{\mathbf{q}}^b_c) + \mathcal{R}^\top(\hat{\mathbf{q}}^b_c)\mathcal{R}(\hat{\mathbf{q}}^b_n)\mathcal{R}(\mathbf{q}^n_{br})
\end{bmatrix}.$$

4) Visual Odometry Orientation Model: Using Figure 4 we can also find a model for the view-matching rotation $\mathbf{q}^c_{cr}$:

$$\mathbf{h}_{vq} = \mathbf{q}^c_{cr} + \eta_{vq} = \mathbf{q}^b_c \otimes \mathbf{q}^n_{br} \otimes \mathbf{q}^b_n \otimes {\mathbf{q}^b_c}^{-1} + \eta_{vq}, \qquad (38)$$

$$\hat{\mathbf{h}}_{vq} = \hat{\mathbf{q}}^c_{cr} = \hat{\mathbf{q}}^b_c \otimes \hat{\mathbf{q}}^n_{br} \otimes \hat{\mathbf{q}}^b_n \otimes {\hat{\mathbf{q}}^b_c}^{-1}. \qquad (39)$$

The analytic residual can be computed two ways, using subtraction [25] or multiplication [29]. We have found the methods to be roughly equivalent in practice, but we currently use the subtraction method.

The analytic model of the residual is

$$\Delta\mathbf{h}_{vq} = \mathbf{q}^c_{cr} + \eta_{vq} - \hat{\mathbf{q}}^c_{cr}.$$

If we multiply each side by the matrix $\Xi^\top(\hat{\mathbf{q}}^c_{cr})$, where

$$\Xi^\top(\mathbf{q}^y_x) = \Xi^\top(\mathbf{q}) = \begin{bmatrix} q_4\mathbf{I} + \lfloor q_{vec}\rfloor & -q_{vec} \end{bmatrix},$$

we can eliminate $\hat{\mathbf{q}}^c_{cr}$ (note that for any quaternion $\mathbf{q}^y_x$, $\Xi^\top(\mathbf{q}^y_x)\mathbf{q}^y_x = \mathbf{0}$),

$$\tilde{\Delta\mathbf{h}}_{vq} = \Xi^\top(\hat{\mathbf{q}}^c_{cr})\Delta\mathbf{h}_{vq} = \Xi^\top(\hat{\mathbf{q}}^c_{cr})\left(\mathbf{q}^c_{cr} + \eta_{vq}\right) - \mathbf{0}. \qquad (40)$$

Which, expanded out, is

$$\begin{aligned}
&\tilde{\Delta\mathbf{h}}_{vq} = \qquad (41) \\
&\Xi^\top(\hat{\mathbf{q}}^c_{cr})\left(\delta\mathbf{q}^b_c \otimes \hat{\mathbf{q}}^b_c \otimes \hat{\mathbf{q}}^n_{br} \otimes \delta\mathbf{q}^b_n \otimes \hat{\mathbf{q}}^b_n \otimes {\hat{\mathbf{q}}^b_c}^{-1} \otimes \delta{\mathbf{q}^b_c}^{-1}\right).
\end{aligned}$$

The analytic Jacobian of (41) is too large to be used in practice. We were able to calculate an approximate Jacobian at nominal values of the quaternions and use that with success. Note that using this method, we calculate the residual in the algorithm using (40) instead of (25), where $\hat{\mathbf{q}}^c_{cr}$ is computed using (39) and $\mathbf{q}^c_{cr}$ is the measurement from the VO. Additionally, the covariance $\mathbf{R}_{vq}$ must be modified by $\Xi^\top(\hat{\mathbf{q}}^c_{cr})$, so that

$$\tilde{\mathbf{R}}_{vq} = \Xi^\top(\hat{\mathbf{q}}^c_{cr})\mathbf{R}_{vq}\Xi(\hat{\mathbf{q}}^c_{cr}).$$

5) Delayed View-Matching Updates: An additional challenge with the view-matching measurement updates is that they are delayed. The stochastic delay is due to the requisite image-processing time. Consequently, computing the position and orientation measurement updates requires a few additional steps. First, the state and covariance must first be restored to their values at the time the image was taken. Then the measurement updates can be applied in the normal fashion, (25) through (31). Finally, the state and covariance must be repropagated back to the current time by re-applying the prediction and measurement updates using the gyroscope, altimeter and accelerometer measurements at their respective timesteps. The state, covariance, IMU, and altimeter information are saved at each timestep to accommodate this requirement.

E. Augment and Marginalize the Relative State

When a new node is created by the view-matching algorithm, the relative portions of the state and covariance must change. This is slightly more difficult as we now use a quaternion to encode the orientation and the yaw is relative. However, the process is straightforward. We use the relationship between quaternions and Euler angles to adjust the state and covariance appropriately.

We can augment and marginalize simultaneously as we are not augmenting additional states or removing unnecessary ones, but are simply replacing the old relative states with new relative states. The states that change are the positions in $\hat{\mathbf{p}}^n$ and the yaw contained in $\hat{\mathbf{q}}^b_n$. The positions are simply replaced with zeros. The quaternion in the state must maintain the same pitch and roll but the yaw must be zeroed out. This is done using the transformations to and from Euler angles. We use the current quaternion $\hat{\mathbf{q}}^b_n$ to find the current pitch $\hat{\theta}$ and roll $\hat{\phi}$ angles

$$\hat{\phi} = \arctan\left(\frac{2\left(\hat{q}_w\hat{q}_x + \hat{q}_y\hat{q}_z\right)}{\hat{q}^2_w + \hat{q}^2_z - \hat{q}^2_x - \hat{q}^2_y}\right) \qquad (42)$$

$$\hat{\theta} = \arcsin\left(2\left(\hat{q}_w\hat{q}_y - \hat{q}_x\hat{q}_z\right)\right), \qquad (43)$$

and then we initialize the new quaternion using

$$\hat{q}_x^+ = \cos\left(\frac{\hat{\theta}}{2}\right)\sin\left(\frac{\hat{\phi}}{2}\right) \tag{44}$$

$$\hat{q}_y^+ = \sin\left(\frac{\hat{\theta}}{2}\right)\cos\left(\frac{\hat{\phi}}{2}\right) \tag{45}$$

$$\hat{q}_z^+ = -\sin\left(\frac{\hat{\theta}}{2}\right)\sin\left(\frac{\hat{\phi}}{2}\right) \tag{46}$$

$$\hat{q}_w^+ = \cos\left(\frac{\hat{\theta}}{2}\right)\cos\left(\frac{\hat{\phi}}{2}\right). \tag{47}$$

Initializing the covariance for the quaternion is slightly different, since the covariance of the quaternion is actually the covariance of the attitude error vector.

The error in yaw must be zeroed-out, but the error in roll and pitch must remain unchanged. Thus, we can use (42)-(47) to accomplish this, but we replace the quaternion $\hat{\mathbf{q}}_n^b$ with the error quaternion $\delta\mathbf{q}_n^b$. Recall that the error quaternion is related to the attitude error vector using (14).

The equations for the new error quaternion given the old error quaternion are obtained by substituting (42) and (43) into (44)-(47). We can simplify the equations by canceling out all the second-order terms, as the elements of $\delta\theta$ are small:

$$\delta q_x^+ \approx \cos\left(\frac{1}{2}\arcsin\left(\delta\theta_y\right)\right)\sin\left(\frac{1}{2}\arctan\left(\frac{\delta\theta_x}{1}\right)\right)$$
$$\delta q_y^+ \approx \sin\left(\frac{1}{2}\arcsin\left(\delta\theta_y\right)\right)\cos\left(\frac{1}{2}\arctan\left(\frac{\delta\theta_x}{1}\right)\right)$$
$$\delta q_z^+ \approx -\sin\left(\frac{1}{2}\arcsin\left(\delta\theta_y\right)\right)\sin\left(\frac{1}{2}\arctan\left(\frac{\delta\theta_x}{1}\right)\right).$$

We need the Jacobian $\mathbf{H}_{\delta\theta}$ of the above three equations with respect to $\delta\theta_x$, $\delta\theta_y$ and $\delta\theta_z$

$$\mathbf{H}_{\delta\theta} = \begin{bmatrix} \frac{\partial\left(\delta q_x^+\right)}{\partial\left(\delta\theta_x\right)} & \frac{\partial\left(\delta q_x^+\right)}{\partial\left(\delta\theta_y\right)} & \frac{\partial\left(\delta q_x^+\right)}{\partial\left(\delta\theta_z\right)} \\ \frac{\partial\left(\delta q_y^+\right)}{\partial\left(\delta\theta_x\right)} & \frac{\partial\left(\delta q_y^+\right)}{\partial\left(\delta\theta_y\right)} & \frac{\partial\left(\delta q_y^+\right)}{\partial\left(\delta\theta_z\right)} \\ \frac{\partial\left(\delta q_z^+\right)}{\partial\left(\delta\theta_x\right)} & \frac{\partial\left(\delta q_z^+\right)}{\partial\left(\delta\theta_y\right)} & \frac{\partial\left(\delta q_z^+\right)}{\partial\left(\delta\theta_z\right)} \end{bmatrix}.$$

We then modify the covariance $\hat{\mathbf{P}}$ using the overall Jacobian of the change in the error state

$$\mathbf{H}_A = \begin{bmatrix} \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times14} \\ \mathbf{0}_{3\times3} & \mathbf{H}_{\delta\theta} & \mathbf{0}_{3\times14} \end{bmatrix}, \tag{48}$$

and the following matrices

$$\mathbf{C} = \mathbf{H}_A\mathbf{P}\mathbf{H}_A^\top$$
$$\mathbf{T} = \mathbf{H}_A\mathbf{P}$$
$$\mathbf{L} = \mathbf{P}\mathbf{H}_A^\top.$$

The matrix $\mathbf{T} : 6 \times 20$ is used to overwrite the top six rows of the covariance. The matrix $\mathbf{L} : 20 \times 6$ overwrites the left-most six columns of the covariance, and the matrix $\mathbf{C} : 6 \times 6$ overwrites the top left corner, in that order. The remaining portions of the covariance are left the same as they are not relative.

IV. Experimental Setup and Results

The results for three scenarios are presented. The first are of the relative MEKF running in real time during a manually-controlled flight. The second set are from an autonomous flight with the estimates in the control loop during a commanded hover. The third set of results are from an autonomous flight in which the hexacopter is following planned paths through an environment. All the flights were completed in a motion capture environment. The motion capture data is only used as the truth for the comparison of the results and to initialize the starting location of the vehicle for easier comparison between truth and estimates.

We utilize a Mikrokopter hexacopter vehicle that carries a Intel Core i7 processor computer to do all the computations, as shown in Figure 1. The computer is running Ubuntu 12.04 Linux and all the applications are implemented in C++ and connected using the robot operating system (ROS) [30]. RGB-D imagery for the VO is provided by an ASUS Xtion Pro Live at 15 Hz. We also use a MicroStrain 3DM-GX3-15 IMU and a LV-MaxSonar-EZ3 sonar altimeter. Data is received from these sensors at 100 Hz and 40 Hz respectively.

A. Real-Time Results

Figures 5 through 8 illustrate the performance of the relative MEKF estimator compared to truth data from the motion capture system during a manually-controlled flight. Velocity truth was generated using a time difference of the 100 Hz motion capture position information, expressed in the body-fixed coordinate frame. The estimates were computed in real time, onboard the aircraft during the manual flight.

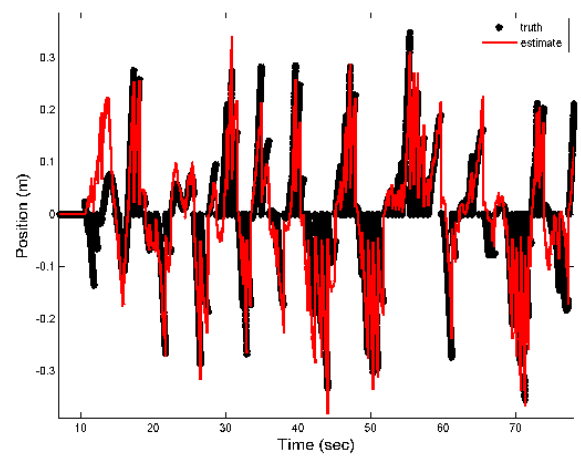

Fig. 5. Relative right position r truth and estimates. This data is produced using the IMU, altimeter, and VO measurements from data received during a manually-controlled flight. The discontinuities in the plots are due to new nodes being created, causing the truth and the estimates to jump to the new relative position. There are not, however, jumps in the global position estimates. We express the global truth from the motion capture in the relative node coordinate frame for the comparison of these results. Results for the relative front and down positions are similar.

In Figure 5 we see the results for the relative state r. Each time a new node is declared by the view-matching algorithm, the relative portions of the state are marginalized out and new relative state elements are augmented in. During this flight, this occurred 85 times. The large discontinuities depicted in Figure 5 are the result of this marginalization and augmentation process.

The body-fixed frame velocity v results are depicted in Figure 6. Notice the lack of discontinuities, as this state is not relative. This demonstrates the ability of the filter to maintain the quality of the non-relative states during the many changes of reference that occur for the relative states. Here we also highlight the advantages of using the improved dynamic model and the benefits that the accelerometer measurement updates provide to the velocity states.

The x component of the quaternion $\mathbf{q}_n^b$, depicted in Figure 7, roughly corresponds to the roll angle of the hexacopter for this flight. Attitude errors are almost always sub-degree in pitch and roll

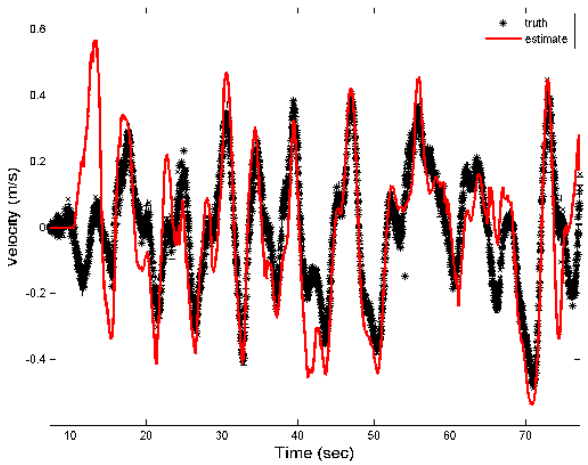

Fig. 6. Body-fixed frame side velocity v truth and estimate comparison. Notice that there are no discontinuities, as the body-frame velocity is not relative. Results for the front and down body-fixed velocities are similar.

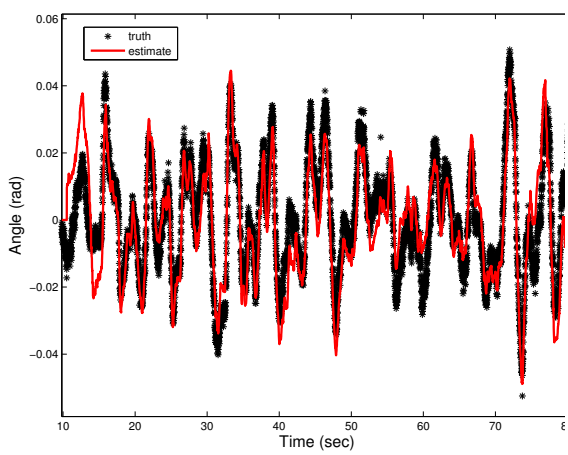

Fig. 7. The x component of the quaternion $\mathbf{q}_n^b$ truth and estimate comparison. There are not any discontinuities in this figure, as this portion of the quaternion is not relative. Results for the other portions of the quaternion are similar.

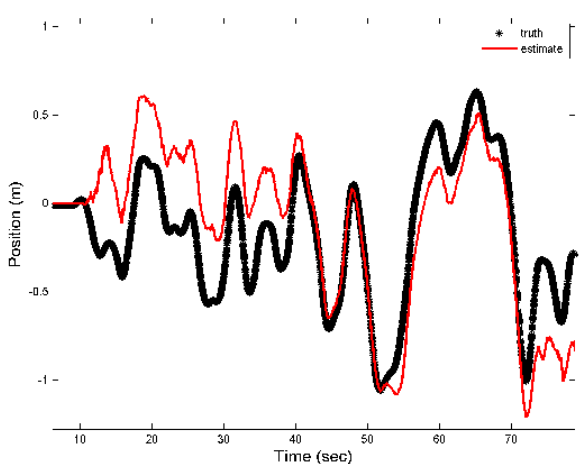

Fig. 8. A comparison between truth and estimates for the global value for east. This estimate is obtained by a vector sum of the edges and the current state at each time-step. This information is not required for the algorithms, but it is helpful in quantifying the quality of the estimates. Results for global north and down position are similar.

compared to truth data. We do have the unavoidable drift in yaw as we do not measure a global yaw angle. This could be corrected with the use of loop closures.

The composition of the relative states and the vector chain of edges to the origin provide the global estimate shown in Figure 8. At this point, no loop closure and optimization have been applied to these results. These are dead-reckoning results using a MEMs-grade IMU and odometry, through the VO. The estimates trend well with the truth information. The global information is not required in the filter and these results are provided for information only.

TABLE I
RMS ERRORS IN STATE AND GLOBAL POSITIONS FROM ESTIMATES PRODUCED DURING A MANUALLY-CONTROLLED FLIGHT. MOTION CAPTURE DATA IS USED AS TRUTH FOR ERROR COMPUTATION. STATE ESTIMATES WERE PRODUCED IN REAL TIME DURING THE FLIGHT.

RMS Error in State Estimates	
State	**RMS Error**
front relative position (f)	0.044 (m)
right relative position (r)	0.048 (m)
down relative position (d)	0.029 (m)
x quaternion (q_x)	0.008 (rad)
y quaternion (q_y)	0.010 (rad)
z quaternion (q_z)	0.051 (rad)
w quaternion (q_w)	0.017 (rad)
forward body-fixed velocity (u)	0.086 (m/s)
side body-fixed velocity (v)	0.147 (m/s)
down body-fixed velocity (w)	0.057 (m/s)
global north position (n)	0.548 (m)
global east position (e)	0.325 (m)
global down position (d)	0.061 (m)

Figures 5 through 8 provide a sample visual confirmation of the quality of the results that the presented filter can produce. Table I below provides RMS error calculations for the state estimates over the same manual flight. These results confirm the performance of the states not shown in the figures along with those that are shown.

B. Estimates in Control Feedback: Hover

The vehicle was commanded to hover at a spot one meter above the take-off location for these results. The controller is based off a previous design [31] with some modifications to account for the relative coordinate systems described above.

Table II provides the RMS errors of all the states and the global positions during the hover flight. Notice that the performance has improved compared to that of Table I. We believe that this is due to several reasons. The first is that the vehicle was commanded to hover and many more VO measurements compared to the same keyframe were made, which allowed more time for filter convergence. Secondly, since the control is based on the estimates, any deviations affect the control and cause motion excitation that increases the observability of that state; this is especially applicable for the attitude and velocity states. Finally, during the manual flight, faster speeds were obtained which could have contributed to more motion blur or other vision artifacts that could decrease the accuracy of the VO measurements.

Figure 9, created based on a similar figure in [32], demonstrates the results from the proposed navigation approach with the current state of the art in estimates-in-the-loop hover performance. Here we see that the proposed approach is close to state-of-the-art performance when compared to any sensor and performs better in north and east by a significant margin than either of the other methods that use a camera.

TABLE II
THE RMS ERRORS IN THE STATE FROM ESTIMATES PRODUCED DURING THE ESTIMATES-IN-THE-LOOP HOVER FLIGHT.

RMS Error for Estimates	
State	**RMS Error**
front relative position (f)	0.019 (m)
right relative position (r)	0.037 (m)
down relative position (d)	0.092 (m)
x quaternion (q_x)	0.006 (rad)
y quaternion (q_y)	0.009 (rad)
z quaternion (q_z)	0.041 (rad)
w quaternion (q_w)	0.017 (rad)
forward body-fixed velocity (u)	0.071 (m/s)
side body-fixed velocity (v)	0.061 (m/s)
down body-fixed velocity (w)	0.051 (m/s)
global north position (n)	0.154 (m)
global east position (e)	0.099 (m)
global down position (d)	0.049 (m)

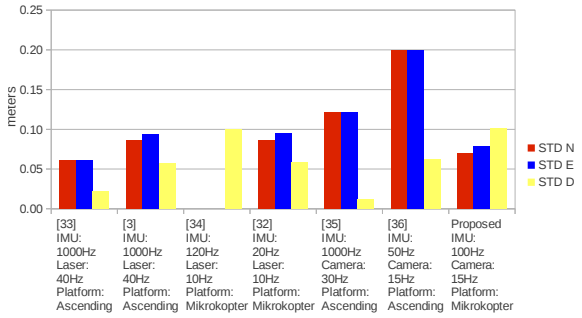

Fig. 9. A comparison of the standard deviations of estimates-in-the-loop hover performance for several different platforms. This data is based on a similar figure published in [32] using material from [33], [3], [34], [32], [35], [36]. Standard deviations are displayed in the different north, east, and down directions, hence lower numbers denote better performance. Note that the proposed estimation approach is competitive with the state of the art in the north and east directions. Platforms are provided by either MikroKopter or Ascending Technologies.

C. Estimates in Control Feedback: Waypoint Path

Figures 10 through 14 demonstrate the results of some of the states during a flight with the state estimates in the control loop. The vehicle avoided obstacles by following paths generated by an onboard path planning algorithm. Goal locations for the vehicle were given as inputs from a remote operator.

Figure 10 provides a comparison of the true and estimated global 3D path of the vehicle. Recall that global information is not a necessary input to the filter. These dead-reckoning results, created using a MEMs-grade IMU and VO (through the MEKF), are provided for information only. In Figure 11 we see the results for the relative forward position state f. There were 26 new nodes created during this autonomous flight. The body frame velocity u results are depicted in Figure 12. The estimates track the truth, even though the magnitude of the speed is small. The y and z components of the quaternion $\mathbf{q}_n^b$ are shown in Figures 13 and 14. The y quaternion roughly corresponds to the pitch angle of the hexacopter for this flight and the z component is close to the yaw angle of the vehicle. The yaw depicted in Figure 14 is the global yaw.

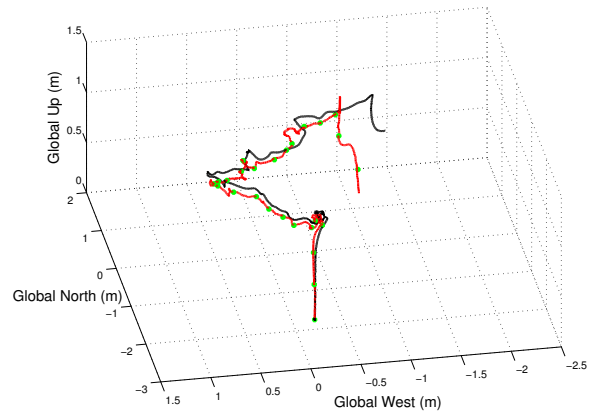

Fig. 10. A comparison between truth and estimates for the global 3D path. This estimate is obtained by a vector sum of the edges and the current state at each time-step. This information is not required for the algorithms, but it is helpful in quantifying the quality of the estimates. The green dots in the figure denote locations of the keyframes. The hexacopter maneuvers around some obstacles to achieve a goal location selected by a remote operator.

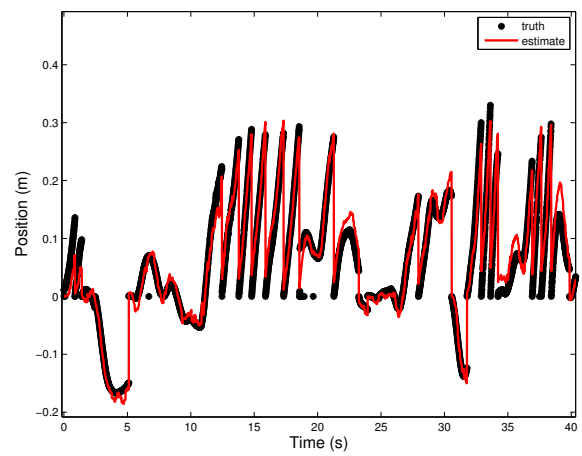

Fig. 11. Relative front position f truth and estimates. This data is from an autonomous flight with estimates in the control loop. The discontinuities in the plots are due to new nodes being created, causing the truth and the estimates to jump to the new relative position. The global positions do not jump when new nodes are created. We express the global truth from the motion capture in the relative node coordinate frame for the comparison of these results. Results for the relative right and down positions are similar.

V. CONCLUSIONS

A relative, vision-based framework, like the approach described here, is an important step in furthering the capabilities of indoor aerial navigation. Current approaches that require globally referenced states often suffer deficiencies from the need for additional state elements to incorporate relative measurements, waiting periods to process global consistency, computation demands on ground stations, or schemes to accommodate large jumps in pose when loop closures are applied. Utilizing a relative approach allows more flexibility as the critical real-time processes of localization and control do not depend on computationally demanding optimization and loop-closure processes. Additionally, relative exteroceptive measurement updates are supported natively in the proposed MEKF and front-facing keyframes provide a rich source of information for path planning.

We have shown the viability and accuracy of the proposed relative MEKF through hardware results comparing estimates to truth

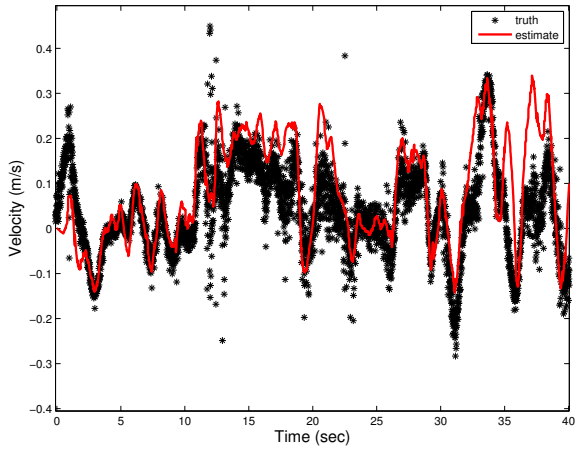

Fig. 12. Body-fixed frame forward velocity u truth and estimate comparison. Notice that there are no discontinuities, as the body-frame velocity is not relative. Results for the right and down body-fixed velocities are similar.

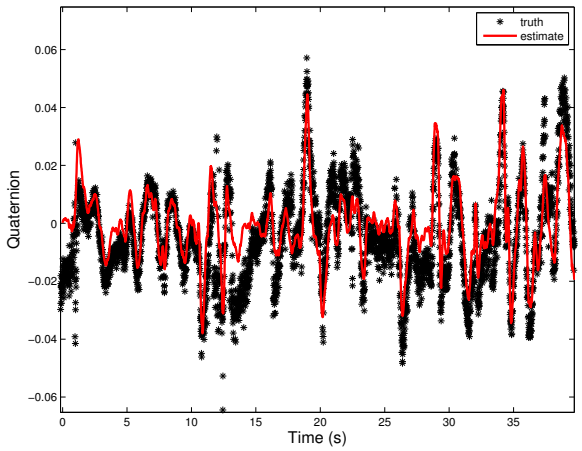

Fig. 13. The y component of the quaternion $\mathbf{q}_n^b$, which is approximately the pitch angle of the hexacopter, comparison of truth and estimates. There are not any discontinuities, as this portion of the quaternion is not relative.

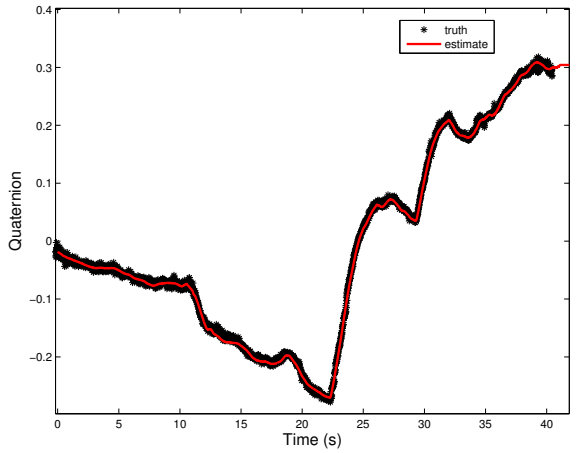

Fig. 14. The z component of the quaternion $\mathbf{q}_n^b$ truth and estimate comparison. This is approximately equal to the yaw angle for this flight. Note that there is some drift as we cannot measure the global yaw angle. We do not show the relative yaw angle in these results, even though that is what is actually tracked in the state vector.

information. We show that we can reliably estimate relative states, marginalize out states, augment new states when a new node is created, and smoothly estimate non-relative states during flights. The estimates are robust enough for utilization in feedback control. The demonstrated feedback-controlled hover results exceed the capabilities of contemporary vision-based approaches.

Acknowledgments

This work was supported through the DoD SMART Scholarship program.

References

[1] A. Bachrach, R. He, and N. Roy, "Autonomous Flight in Unstructured and Unknown Indoor Environments," in *Proc. of the EMAV Conference*. European Micro Air Vechicle, Sept. 2009, pp. 2–9.

[2] M. Blosch, S. Weiss, D. Scaramuzza, and R. Siegwart, "Vision based MAV navigation in unknown and unstructured environments," in *Proc. IEEE Int. Conf. on Robotics and Automation*, 2010, pp. 21–28.

[3] S. Shen, N. Michael, and V. Kumar, "Autonomous multi-floor indoor navigation with a computationally constrained MAV," in *IEEE Intl. Conf. on Robotics and Automation*, May 2011, pp. 20–25.

[4] A. S. Huang, A. Bachrach, P. Henry, M. Krainin, D. Maturana, D. Fox, and N. Roy, "Visual Odometry and Mapping for Autonomous Flight Using an RGB-D Camera," in *Int. Symposium on Robotics Research*, Flagstaff, Arizona, USA, 2011.

[5] A. Bachrach, S. Prentice, N. Roy, R. He, A. D. Winter, and G. Hemann, "RANGE: Robust autonomous navigation in GPS-denied environments," *Journal of Field Robotics*, vol. 28, no. 5, pp. 644–666, Sept. 2011.

[6] M. W. Achtelik, S. Lynen, S. Weiss, L. Kneip, M. Chli, and R. Siegwart, "Visual-inertial SLAM for a small helicopter in large outdoor environments," *IEEE International Conference on Intelligent Robots and Systems*, pp. 2651–2652, Oct. 2012.

[7] F. Fraundorfer, L. Heng, D. Honegger, G. H. Lee, P. Tanskanen, and M. Pollefeys, "Vision-Based Autonomous Mapping and Exploration Using a Quadrotor MAV," in *IEEE Intl. Conf. on Intelligent Robots and Systems*, 2012.

[8] T. Tomic, K. Schmid, P. Lutz, M. Kassecker, E. Mair, I. Grixa, F. Ruess, M. Suppa, and D. Burshka, "Toward a Fully Autonomous UAV: Research Platform for Indoor and Outdoor Urban Search and Rescue," *Robotics and Automation Magazine*, no. September, 2012.

[9] S. Weiss, M. Achtelik, S. Lynen, M. Chli, and R. Siegwert, "Real-time onboard visual-inertial state estimation and self-calibration of mavs in unknown environments," in *IEEE Intl. Conf. Robotics and Automation*, 2012.

[10] D. Nister, O. Naroditsky, and J. Bergen, "Visual odometry," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition*, vol. 1, 2004.

[11] G. Klein and D. Murray, "Parallel tracking and mapping for small AR workspaces," in *IEEE and ACM Int. Symposium on Mixed and Augmented Reality*, Washington, DC, USA, 2007, pp. 1–10.

[12] S. Roumeliotis and J. Burdick, "Stochastic cloning: a generalized framework for processing relative state measurements," in *IEEE Intl. Conf. on Robotics and Automation*, vol. 2, no. May, 2002, pp. 1788–1795.

[13] M. Achtelik and A. Bachrach, "Autonomous navigation and exploration of a quadrotor helicopter in GPS-denied indoor environments," *Robotics: Science and Systems*, 2009.

[14] R. C. Leishman, J. Macdonald, R. W. Beard, and T. W. McLain, "Quadrotors & Accelerometers State Estimation with an Improved Dynamic Model," *Control Systems Magazine (Submitted; preprint available from the authors)*, 2013.

[15] S. Thrun, "Simultaneous localization and mapping," in *Robotics and cognitive approaches to spatial mapping*. Springer, 2008, pp. 871–889.

[16] G. Grisetti, R. Kummerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based SLAM," *IEEE Intelligent Transportation Systems Magazine*, no. Winter, pp. 31–43, Feb. 2010.

[17] M. Cummins and P. Newman, "FAB-MAP: Probabilistic Localization and Mapping in the Space of Appearance," *The International Journal of Robotics Research*, vol. 27, no. 6, pp. 647–665, June 2008.

[18] R. Leishman, J. Macdonald, R. W. Beard, and T. McLain, "Relative navigation and control of a hexacopter," in *IEEE Intl. Conf. on Robotics and Automation*, St. Paul, MN, USA, May 2012, pp. 4937–4942.

[19] R. C. Leishman, T. W. McLain, and R. W. Beard, "Relative Navigation Approach for Vision-Based Aerial GPS-Denied Navigation," in *Intl. Conference on Unmanned Aircraft Systems (Submitted; preprint available from the authors)*, Atlanta, GA, 2013.

[20] B. Kuipers and Y. Byun, "A robust, qualitative method for robot spatial learning," *Proc. of the AAAI*, 1988.

[21] R. C. Leishman, D. Koch, and T. McLain, "Robust Motion Estimation Using an RBG-D Camera," in *AIAA Infotech @ Aerospace Conference (Submitted; preprint available from the authors)*, 2013.

[22] G. Sibley, C. Mei, I. Reid, and P. Newman, "Planes, trains and automobiles autonomy for the modern robot," in *IEEE Int. Conf. on Robotics and Automation*, May 2010, pp. 285–292.

[23] J. Kuipers, *Quaternions and Rotation Sequences*. Princeton University Press, 1999.

[24] J. L. Crassidis, F. L. Markley, and Y. Cheng, "Survey of Nonlinear Attitude Estimation Methods," *Journal of Guidance, Control, and Dynamics*, vol. 30, no. 1, pp. 12–28, Jan. 2007.

[25] N. Trawny and S. I. Roumeliotis, "Indirect Kalman Filter for 3D Attitude Estimation A Tutorial for Quaternion Algebra," Multiple Autonomous Robotic Systems Lab, Tech. Rep. 612, 2005. [Online]. Available: http://www.cs.umn.edu/ trawny

[26] P. Martin and E. Salaün, "Generalized multiplicative extended kalman filter for aided attitude and heading reference system," in *AIAA Guidance, Navigation, and Control*, no. August, 2010.

[27] R. W. Beard and T. McLain, *Small Unmanned Aircraft*. Princeton University Press, 2012.

[28] J. Macdonald, R. C. Leishman, R. W. Beard, and T. W. McLain, "Analysis of an Improved IMU-Based Observer for Multirotor Helicopters," *Journal of Intelligent & Robotic Systems (in review; preprint available from the authors)*, 2013.

[29] S. Weiss and R. Siegwart, "Real-Time Metric State Estimation for Modular Vision-Inertial Systems," in *IEEE Intl. Conf. on Robotics and Automation*, vol. 231855, Shanghai, May 2011, pp. 4531–4537.

[30] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng, "ROS : an open-source Robot Operating System," in *IEEE Intl. Conf. on Robotics and Automation Workshop on Open Source Robotics*, Kobe, Japan, 2009.

[31] J. Ferrin, R. Leishman, R. Beard, and T. McLain, "Differential Flatness Based Control of a Rotorcraft For Aggressive Maneuvers," in *IEEE Int. Conf. Intelligent Robots and Systems*, 2011.

[32] I. Sa and P. Corke, "System Identification, Estimation and Control for a Cost Effective Open-Source Quadcopter," in *IEEE Int. Conf. on Robotics and Automation*, 2012, pp. 2202–2209.

[33] A. Bachrach, "Autonomous Flight in Unstructured and Unknown Indoor Environments," Ph.D. dissertation, MIT, 2009.

[34] S. Grzonka, G. Grisetti, and W. Burgard, "Towards a navigation system for autonomous indoor flying," in *IEEE Intl. Conf. Robotics and Automation*, no. Section III, May 2009, pp. 2878–2883.

[35] M. Achtelik, M. Achtelik, S. Weiss, and R. Siegwart, "Onboard IMU and monocular vision based control for MAVs in unknown in- and outdoor environments," *IEEE Intl. Conf. on Robotics and Automation*, pp. 3056–3063, May 2011.

[36] F. Bourgeois, L. Kneip, S. Weiss, and R. Siegwart, "Delay and Dropout Tolerant State Estimation for MAVs," in *Proc. Intl. Symposium on Experimental Robotics*, Berlin, 2010, pp. 1–14.